

TransitSync

```
./execute --routing "Arrival-Time & Smart Connections"
```

\$ The Problem

> RIGID SEARCH CONSTRAINTS

Platforms like IRCTC only allow searches by Departure Date, ignoring the passenger's actual deadline.

> MANUAL ROUTE PATCHING

If a direct train is unavailable or mistimed, users must guess hubs and manually piece together tickets.

> HIGH RISK LAYOVERS

Manually calculating safe buffer times between trains is stressful and prone to costly errors.



\$ The Solution

Intelligent Transit Graphs

TransitSync flips the paradigm: you tell us **when you need to arrive**, and we compute the math backwards.

Our graph-based algorithm automatically computes multi-leg journeys ($A \rightarrow C \rightarrow B$) when direct routes are unavailable, instantly uncovering hidden possibilities.



\$ Core Features



ARRIVAL-FIRST SEARCH

Input a strict arrival deadline. The engine filters all possible permutations to ensure you reach your destination on time.



SMART AUTO-CONNECT

Seamlessly splices different trains at logical transit hubs to create viable itineraries when direct routes are sold out.



BUFFER OPTIMIZATION

Automatically enforces safe layover windows (e.g., min 45 mins) to protect against historical delay data of the first leg.

\$ How It Works

2_Graph Traversal

Engine searches network for direct & 1-layover paths.

4_Result Ranking

Final itineraries ranked by travel time and low friction.

1_Parameter Input

User defines Source, Dest, Date, and critical Arrival Time.

3_Logic Filtering

Routes are pruned based on buffers & seat availability.

\$ **Projected Impact**

[**85%**]

REDUCED SEARCH TIME

Eliminates the hours spent cross-referencing multiple tabs and timetables manually.

[**3x**]

MORE VIABLE ROUTES

By unlocking 1-stop connections, users have significantly higher chances of securing a ticket.

System

Architecture

\$ Tech Stack

FRONTEND PROTOTYPE

A responsive web interface focused on inputting time constraints and visualizing multi-leg journeys.

HTML5

Tailwind CSS

Vanilla JS

ROUTING LOGIC

To scale this, traditional DBs fail. The system requires a graph-based approach to calculate N-depth connections.

Neo4j (Graph DB)

Dijkstra's Algo

Node.js API

System_Halt

Execution finished. Awaiting input queries...

> Hackathon Team 2026_

\$ Image Sources

600 × 400

https://miro.medium.com/v2/resize:fit:1200/1*QqAKIUgEAlHLVr3qlcbYuA.png

Source: muz.li

600 × 400

https://images.stockcake.com/public/e/9/6/e96c2d89-e1f3-4aba-9d80-8971eefcf083_medium/global-network-map-stockcake.jpg

Source: stockcake.com

\$ Image Sources



https://miro.medium.com/v2/resize:fit:1200/1*QqAKIUgEAlHLVr3qlcbYuA.png

Source: muz.li



https://images.stockcake.com/public/e/9/6/e96c2d89-e1f3-4aba-9d80-8971eefcf083_medium/global-network-map-stockcake.jpg

Source: stockcake.com